

¿Qué hace System.arraycopy en Java?

System.arraycopy es un método rápido de Java que sirve para copiar elementos de un arreglo a otro, o incluso dentro del mismo arreglo, de manera eficiente.

Sintaxis

System.arraycopy(origen, posOrigen, destino, posDestino, cantidad);

Parámetros:

1. origen: arreglo del cual se copiarán los datos.
2. posOrigen: índice inicial del arreglo origen.
3. destino: arreglo donde se pegarán los datos.
4. posDestino: índice inicial en el arreglo destino.
5. cantidad: número de elementos a copiar.

Ejemplo básico

```
int[] a = {1, 2, 3, 4, 5};  
int[] b = new int[5];  
System.arraycopy(a, 1, b, 0, 3);
```

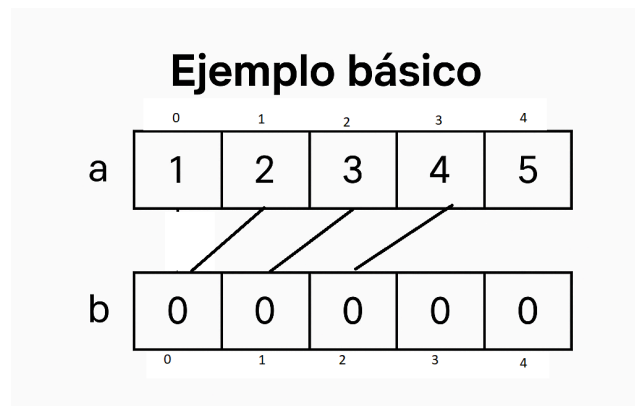
Resultado:

b = {2, 3, 4, 0, 0}

Interpretación:

```
System.arraycopy(a, 1, b, 0, 3);
```

Copia los elementos del arreglo a a partir de la posición 1 hacia el arreglo b, comenzando a llenar desde la posición 0 y copiando tres elementos.



Ejemplo básico

```
int[] a = {1, 2, 3, 4, 5};
```

```
System.arraycopy(a, 0, a, 1, 3);
```

Antes:

[1, 2, 3, 4, 5]

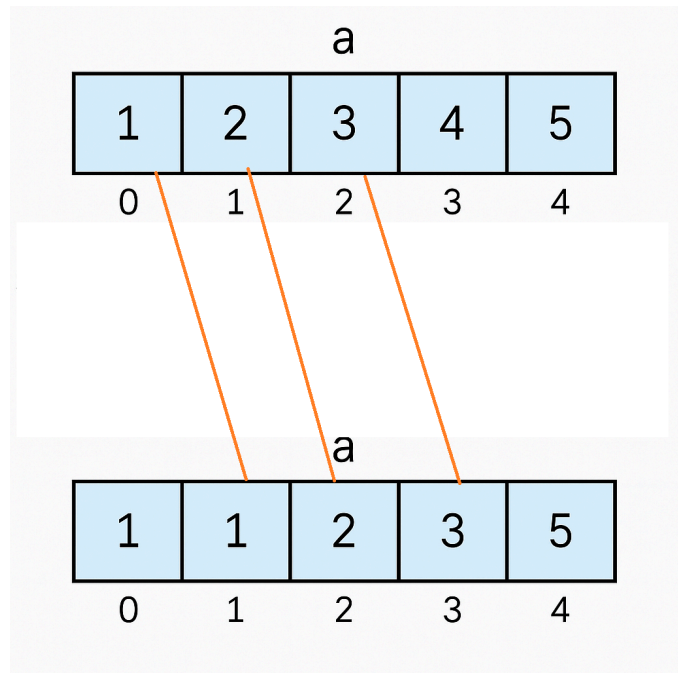
Después:

[1, 1, 2, 3, 5]

Interpretación:

```
System.arraycopy(a, 0, a, 1, 3);
```

Copia los elementos del arreglo a a partir de la posición 1 hacia el mismo arreglo a, comenzando a llenar desde la posición 1 y copiando tres elementos.



¿Por qué se usarlo?

- Es muy rápido (usa código nativo optimizado).
- Maneja correctamente solapamientos.
- Útil para insertar o eliminar elementos al desplazar posiciones.
- Ideal para copiar secciones grandes de datos.

En resumen

`System.arraycopy`:

- Copia porciones de un arreglo a otro.
- Permite copiar dentro del mismo arreglo.
- Es más eficiente que un ciclo `for`.
- Requiere que los tipos de datos sean compatibles.

Manejo de Errores en Java: try-catch, throw y throws

En Java, los conceptos **try-catch**, **throw**, y **throws** están relacionados con el manejo de excepciones, pero cada uno tiene un propósito y se utiliza en contextos distintos. Aquí tienes una explicación clara y diferenciada:

1. try-catch

¿Qué es?

Un bloque utilizado para manejar excepciones que pueden ocurrir durante la ejecución.

Propósito:

Evitar que el programa termine abruptamente, para ello este bloque permite capturar y manejar errores.

Ejemplo:

```
try {  
    int resultado = 10 / 0; // Lanza ArithmeticException  
} catch (ArithmeticException e) {  
    System.out.println("No se puede dividir entre cero.");  
}
```

2. throw

¿Qué es?

Es una instrucción usada para lanzar una excepción de manera explícita.

Propósito:

Su propósito es indicar una condición inválida o problemática detectada por el programador.

Ejemplo:

```
public void validarEdad(int edad) {  
    if (edad < 18) {  
        throw new IllegalArgumentException("La edad debe ser  
mayor o igual a 18.");  
    }  
}
```

3. throws

¿Qué es?

Es una declaración en la firma (nombre) del método que indica que puede lanzar excepciones.

Propósito:

Informar a quien vea el código (programador) que el método puede generar una excepción que debe ser manejada desde afuera (por ejemplo quien lo invoca)

Ejemplo:

```
public void leerArchivo(String ruta) throws IOException {  
    FileReader fr = new FileReader(ruta);  
}  
  
//Alguien debe lanzar la excepcion  
public void procesarArchivo() {  
    try {  
        leerArchivo("datos.txt");  
        System.out.println("Archivo leído correctamente.");  
    } catch (IOException e) {  
        System.out.println("Error al leer el archivo: " +  
e.getMessage());  
    }  
}
```

Diferencias	
try-catch	Maneja excepciones en tiempo de ejecución.
throw	Lanza una excepción en un punto específico.
throws	Declara que un método puede lanzar excepciones.

¿Cuándo usar cada uno?	
try-catch	Cuando se desea manejar el error donde ocurre.
throw	Para lanzar una excepción cuando una <i>condición</i> es inválida.
throws	Cuando se desea delegar el manejo de excepciones al método llamante.

Tabla de Errores y Excepciones en Java con Explicación de Ejemplos

Categoría	Descripción	Ejemplos	¿Qué significa?
Errores (Error)	Problemas graves de la JVM o del entorno. No dependen del programador y generalmente no pueden recuperarse.	OutOfMemoryError	La JVM se quedó sin memoria para crear nuevos objetos. Sucede cuando se crean demasiados objetos o se cargan datos gigantes.
		StackOverflowError	La pila de ejecución se llenó, normalmente por recursión infinita o muy profunda.

Categoría	Descripción	Ejemplos	¿Qué significa?
Checked Exceptions	Excepciones que el compilador obliga a manejar porque representan situaciones externas y esperadas.	IOException	Ocurre cuando falla una operación de entrada/salida: leer archivos, escribir datos, abrir recursos externos.
		SQLException	Se produce cuando algo falla en una operación de base de datos: error de conexión, consulta inválida, tabla inexistente, etc.
		ClassNotFoundException	Ocurre cuando se intenta cargar una clase que no existe o no está disponible en el proyecto.

Categoría	Descripción	Ejemplos	¿Qué significa?
Unchecked Exceptions	Errores de programación. No es obligatorio manejarlos, pero deben evitarse corrigiendo la lógica.	NullPointerException	Sucedee al intentar usar una referencia que es <i>null</i> . Ejemplo: llamar un método de un objeto no inicializado.
		ArrayIndexOutOfBoundsException	Acceder a un índice inválido en un arreglo. Ejemplo: usar un índice mayor al tamaño del arreglo.
		ArithmeticException	Error matemático como dividir entre cero.
		IllegalArgumentException	Un método recibe un argumento inválido, por ejemplo un número negativo donde no debe.